

CS330 Operating Systems and Lab.

Deadlock

Spring 2026



Instructor : Insik Shin

Basic Structure for Synchronization

Thread 1



Sync. Entry

Critical
Section

Sync. Exit

Thread 2



Sync. Entry

Critical
Section

Sync. Exit

Synchronization - Desired Solution

- Mutual exclusion
 - Only one process can be in critical section (CS) at a time
- Progress
 - A process wishing to enter CS **immediately** enters if CS is not occupied or **eventually** enters otherwise
- Bounded waiting
 - A bound should exist on the number of times that other processes enter CS while a process waits for entering CS

Dining-Philosophers Problem

- Philosophers who spend their lives thinking and eating
 - when hungry, they try to pick up two chopsticks (one by one) to eat
 - each chopstick cannot belong to two or more philosophers



Dining-Philosophers Problem

- Philosophers who spend their lives thinking and eating
 - when hungry, they try to pick up two chopsticks (one by one) to eat (atomic operation)
 - each chopstick cannot belong to two or more philosophers (mutual exclusion)



- Shared data
 - Bowl of rice (data set)
 - Semaphore **chopstick** **[5]** initialized to 1

Dining-Philosophers Problem

- The structure of Philosopher i :

```
do {  
    // think  
  
    wait ( chopstick[i] );  
    wait ( chopstick[ (i + 1) % 5] );  
  
    // eat  
  
    signal ( chopstick[i] );  
    signal ( chopstick[ (i + 1) % 5] );  
  
} while (TRUE);
```

- Correctness?
 - Mutual exclusion
 - Progress
 - Bounded waiting time

The Deadlock Problem

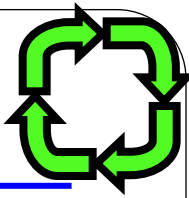
- What is it?
 - Multiple competing processes/threads are waiting for the other to finish, and thus neither ever does
- Intersection
 - No cars can move forward just waiting for the others moving forward
 - Several cars may have to be backed up to resolve a deadlock (preempt resources and rollback)



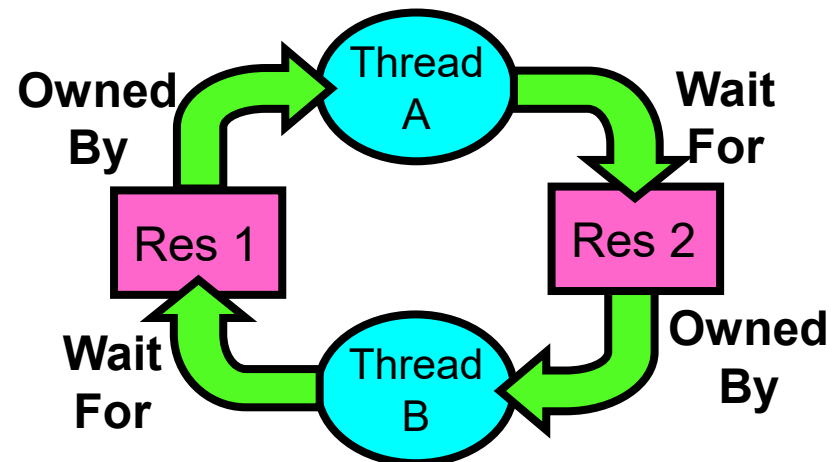
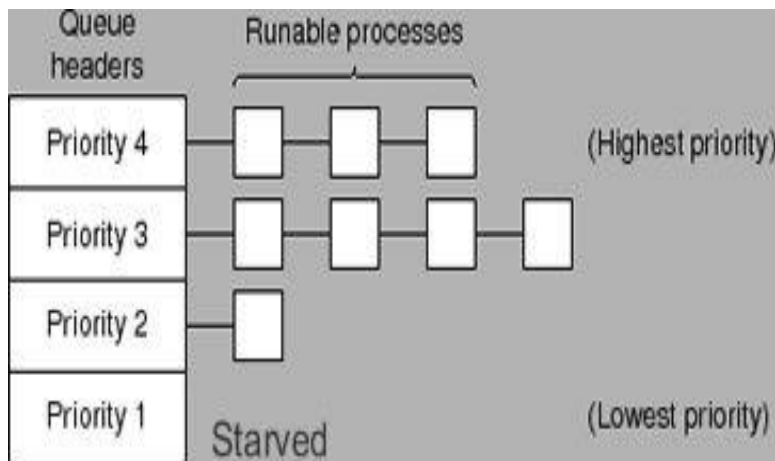
The Deadlock Problem

- What is it?
 - Multiple competing processes/threads are waiting for the other to finish, and thus neither ever does
- When does it happen?
 - a set of blocked processes each holding a resource and waiting to acquire a resource held by another process in the set

Starvation vs Deadlock



- Starvation: thread waits indefinitely
 - Example, low-priority thread waiting for resources constantly in use by high-priority threads
- Deadlock: circular waiting for resources
 - Thread A owns Res 1 and is waiting for Res 2
 - Thread B owns Res 2 and is waiting for Res 1



- Deadlock $\Rightarrow$ Starvation but not vice versa
 - Starvation can end (but doesn't have to)
 - Deadlock can't end without external intervention

Dining-Philosopher Problem

- Solutions to the deadlock problem?
- the structure of Philosopher i :

```
do {  
    // think  
  
    wait ( chopstick[i] );  
    wait ( chopstick[ (i + 1) % 5] );  
  
    // eat  
  
    signal ( chopstick[i] );  
    signal ( chopstick[ (i + 1) % 5] );  
} while (TRUE);
```



Dining-Philosopher Problem

- Solutions to the deadlock problem
 - Allow a philosopher to pick up chopsticks only if both chopsticks are available (needs to be done in a critical section)
 - Allow preemption or sharing
 - Add one more chopstick or disallow one philosopher
 - Use an asymmetric solution; an odd-numbered philosopher picks up a left chopstick and then a right one, whereas an even-numbered philosopher picks up in the other way

Where are we going with synchronization?

Programs	Shared Programs
Higher-level API	Locks Semaphores Monitors Send/Receive
Hardware	Load/Store Disable Ints Test&Set Comp&Swap

- Monitor
 - A higher-level abstraction that ensures a single active process to access semaphores of interest

Basic Structure for Synchronization

Thread 1



Sync. Entry

Critical
Section

Sync. Exit

Thread 2



Sync. Entry

Critical
Section

Sync. Exit

Basic Structure for Synchronization

Thread 1



wait (mutex);

wait (chopstick[0]);
wait (chopstick[1]);

signal (mutex);

Critical
Section

signal (chopstick[0]);
signal (chopstick[1]);

Thread 2



wait (mutex);

wait (chopstick[1]);
wait (chopstick[0]);

signal (mutex);

Critical
Section

signal (chopstick[1]);
signal (chopstick[0]);

Sync.
Entry

Sync.
Exit

Basic Structure for Synchronization

Thread 1



**Sync.
Entry**

wait (mutex);

wait (chopstick[0]);
wait (chopstick[1]);

**Critical
Section**

signal (chopstick[0]);
signal (chopstick[1]);

signal (mutex);

Thread 2



wait (mutex);

wait (chopstick[1]);
wait (chopstick[0]);

**Critical
Section**

signal (chopstick[1]);
signal (chopstick[0]);

signal (mutex);

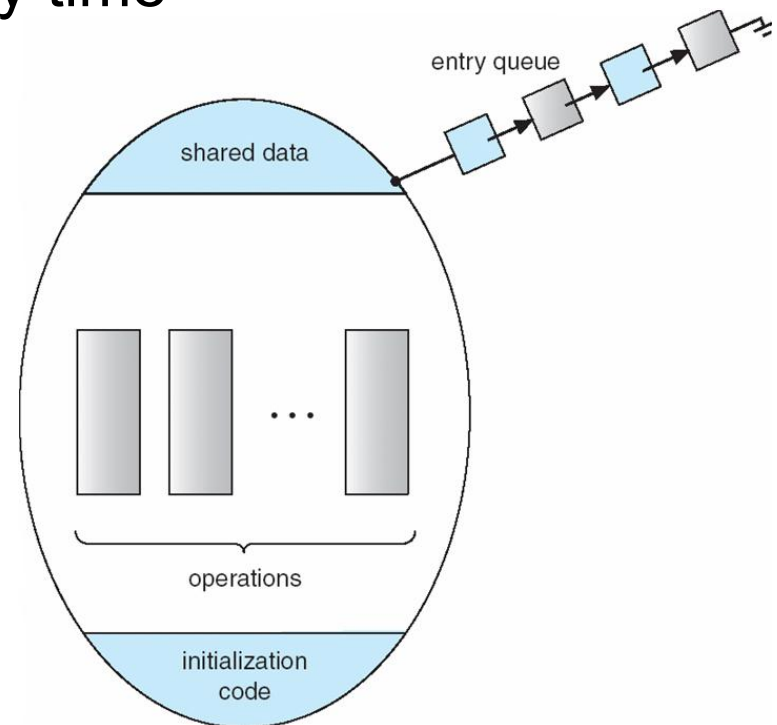
**Sync.
Exit**

Monitors

- A high-level abstraction that provides a convenient and effective mechanism for process (thread) synchronization
- Encapsulates shared data and the operations that manipulate it, ensuring only one process (thread) can be active within the monitor at any time

monitor monitor-name

```
{  
    // shared variable declarations  
    procedure P1 (...) { .... }  
    ...  
    procedure Pn (...) { ..... }  
    Initialization code ( .... ) { ... }  
}
```



Solution to Dining Philosophers (cont)

- Each philosopher invokes the operations `pickup()` and `putdown()` in the following sequence:

- The structure of Philosopher *i*:

```
do {  
    // THINK
```

```
DiningPhilosophers.pickup (i)
```

```
// EAT
```

```
DiningPhilosophers.putdown (i)
```

```
} while (TRUE);
```

Solution to Dining Philosophers

Monitor DiningPhilosophers

```
{
    enum { THINKING; HUNGRY, EATING} state [5] ;

    condition self [5];

    void pickup (int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING) self [i].wait;
    }

    void putdown (int i) {
        state[i] = THINKING;
        // test left and right neighbors
        test((i + 4) % 5);
        test((i + 1) % 5);
    }
}

void test (int i) {
    if ( (state[(i + 4) % 5] != EATING) &&
        (state[i] == HUNGRY) &&
        (state[(i + 1) % 5] != EATING) ) {
        state[i] = EATING ;
        self[i].signal () ;
    }
}

initialization_code() {
    for (int i = 0; i < 5; i++)
        state[i] = THINKING;
}
```

Monitors

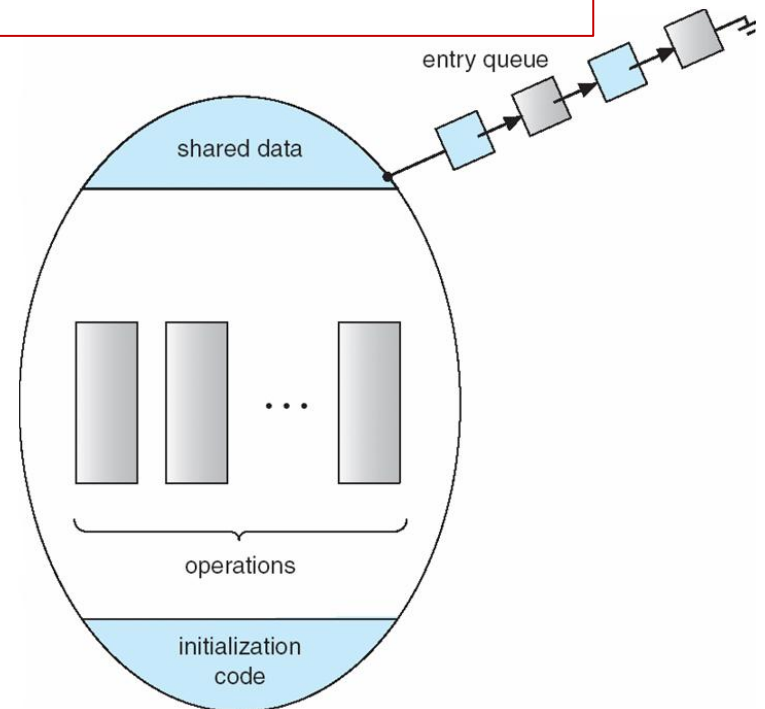
- A high-level abstraction that provides a convenient and effective mechanism for process synchronization
- Only one process may be active within the monitor at a time

How can we make sure this property?

```
monitor monitor-name
{
    // shared variable declarations

    procedure P1 (...) { .... }
    ...
    procedure Pn (...) {.....}

    Initialization code ( ....) { ... }
}
```



Monitor Implementation Using Semaphores

- Variables

```
semaphore mutex; // (initially = 1)
semaphore next;  // (initially = 0)
int next-count = 0;
```

- Each procedure ***F*** will be replaced by

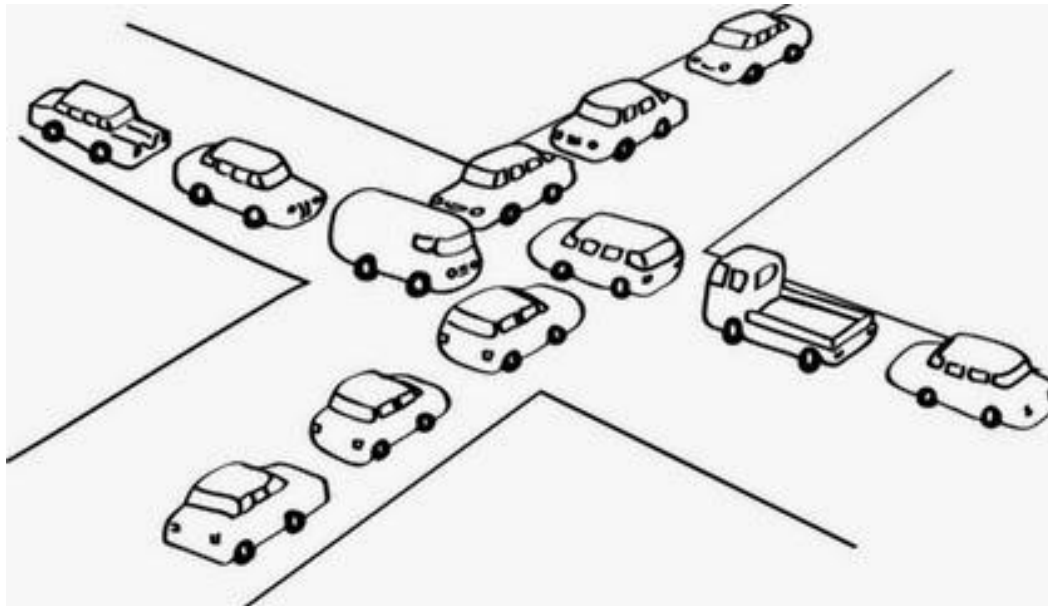
```
wait(mutex);
...
                                body of F;
...
if (next_count > 0) signal(next)
else signal(mutex);
```

- Mutual exclusion within a monitor is ensured.

Feature	Monitor	Semaphore
Level of abstraction	High-level construct	Low-level construct
Mutual exclusion	Built-in automatically	Must be implemented manually using wait/signal
Encapsulation	Encapsulates shared data + operations	No encapsulation — data and logic are separate
Usage complexity	Easier to use — handles locking internally	More error-prone — requires careful use
Thread access	Only one thread can be active in a monitor at a time	Multiple threads can enter if semaphore count allows
Language support	Often language-supported (e.g., Java synchronized)	Typically implemented using library functions or OS support
Purpose	Synchronizes access to shared resources using methods	Controls access based on counting mechanism
Safety	Less prone to errors like deadlocks or forgetting a signal	Higher risk of issues like forgetting to signal()

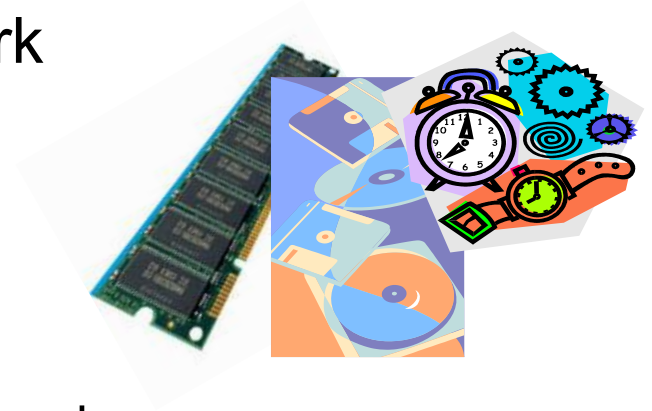
Deadlocks

- Deadlocks happen upon some particular relationships between threads and resources
- Approaches to deadlocks
 - Can we predict a deadlock to happen?
 - Can we prevent deadlocks from happening?



Resources

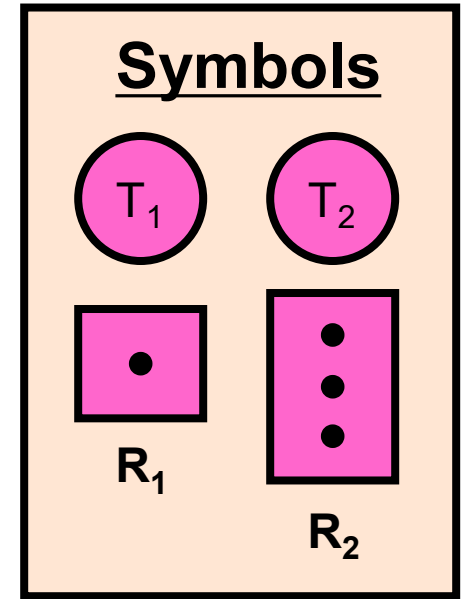
- Threads need resources to do their work
 - Resources: CPU time, disk space, memory
- Two types of resources:
 - **Preemptable** – can take it away
 - CPU, Embedded security chip
 - **Non-preemptable** – must leave it with the thread
 - Disk space, printer, chunk of virtual address space
 - Critical section
- Resources may require **exclusive access** or may be **sharable**
 - Read-only files are typically sharable
 - Printers are not sharable during time of printing
- One of the major tasks of an operating system is to manage resources



Resource-Allocation Graph

- System Model

- A set of Threads $T_1, T_2, \dots, T_n$
- Resource types $R_1, R_2, \dots, R_m$
CPU cycles, memory space, I/O devices
- Each resource type R_i has W_i instances.
- Each thread utilizes a resource as follows:
 - `Request()` / `Use()` / `Release()`



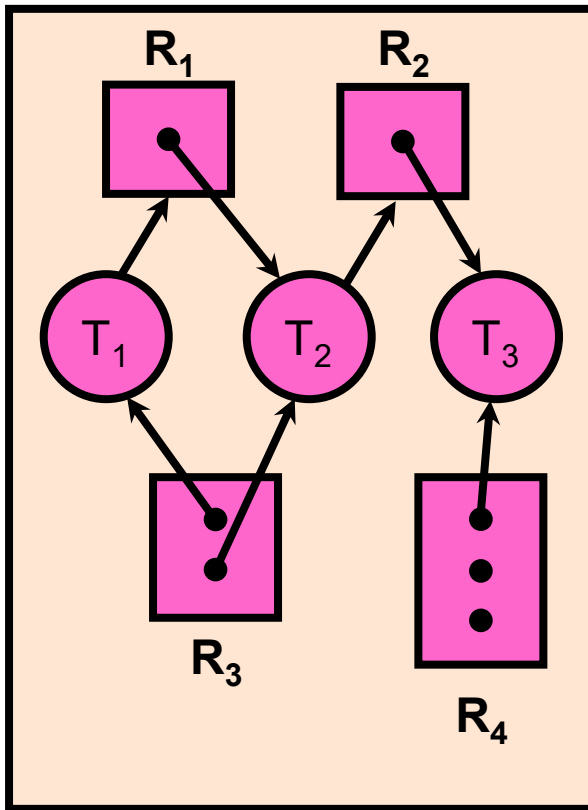
- Resource-Allocation Graph:

- V is partitioned into two types:
 - $T = \{T_1, T_2, \dots, T_n\}$, the set of threads in the system.
 - $R = \{R_1, R_2, \dots, R_m\}$, the set of resource types in system
- **request** edge – directed edge $T_i \rightarrow R_j$
- **assignment** edge – directed edge $R_j \rightarrow T_i$

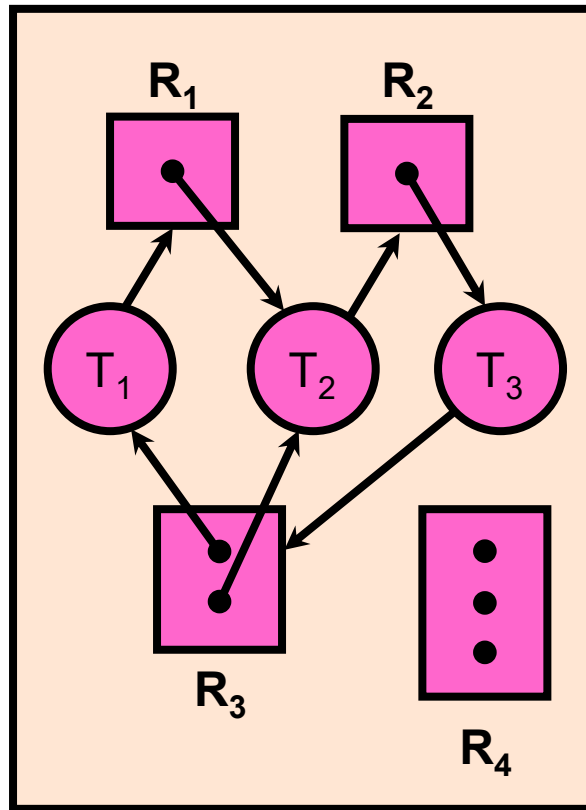
Resource Allocation Graph Examples

- Recall:

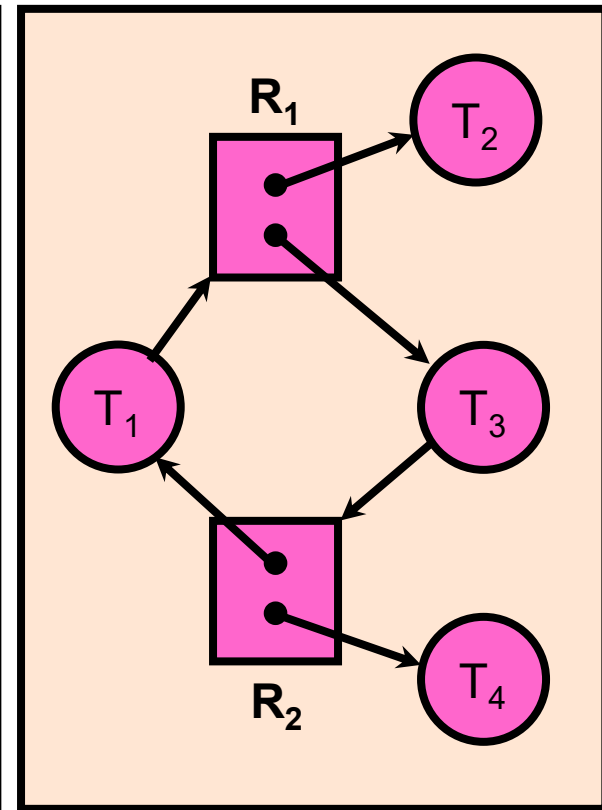
- request edge – directed edge $T_i \rightarrow R_j$
- assignment edge – directed edge $R_j \rightarrow T_i$



Simple Resource
Allocation Graph



Allocation Graph
with Deadlock

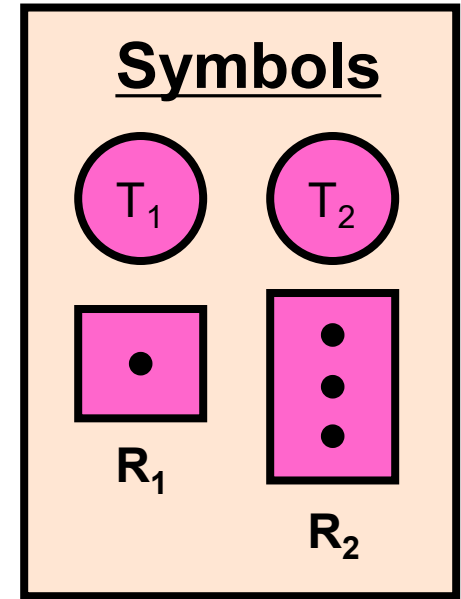


Allocation Graph with
Cycle, but No Deadlock

Resource-Allocation Graph

- System Model

- A set of Threads $T_1, T_2, \dots, T_n$
- Resource types $R_1, R_2, \dots, R_m$
CPU cycles, memory space, I/O devices
- Each resource type R_i has W_i instances.
- Each thread utilizes a resource as follows:
 - `Request()` / `Use()` / `Release()`



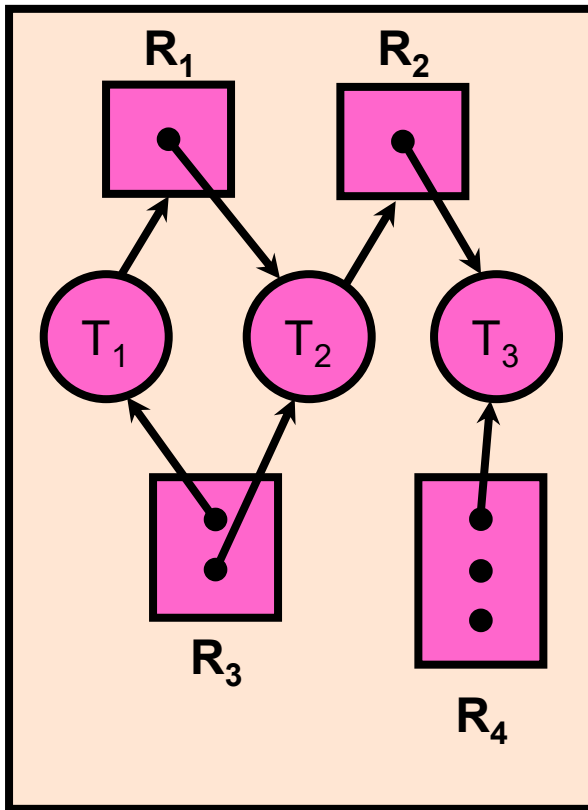
- Resource-Allocation Graph:

- V is partitioned into two types:
 - $T = \{T_1, T_2, \dots, T_n\}$, the set of threads in the system.
 - $R = \{R_1, R_2, \dots, R_m\}$, the set of resource types in system
- **request** edge – directed edge $T_i \rightarrow R_j$
- **assignment** edge – directed edge $R_j \rightarrow T_i$

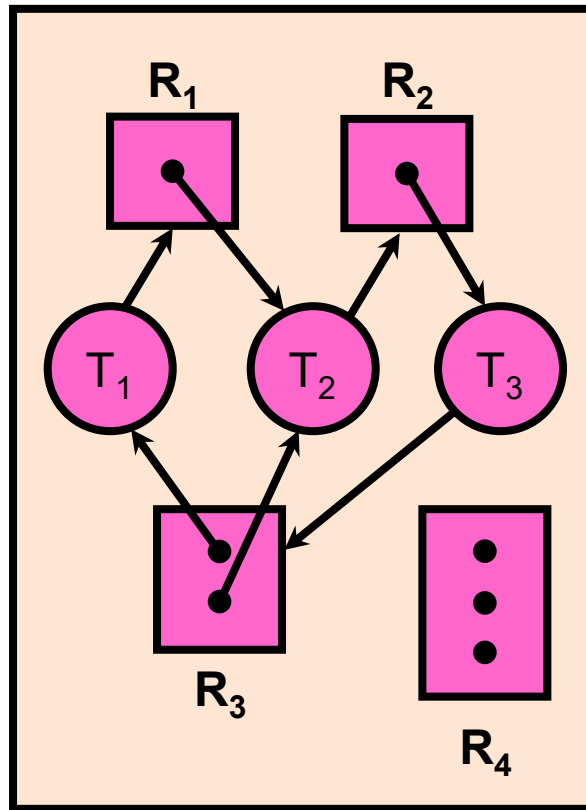
Resource Allocation Graph Examples

- Recall:

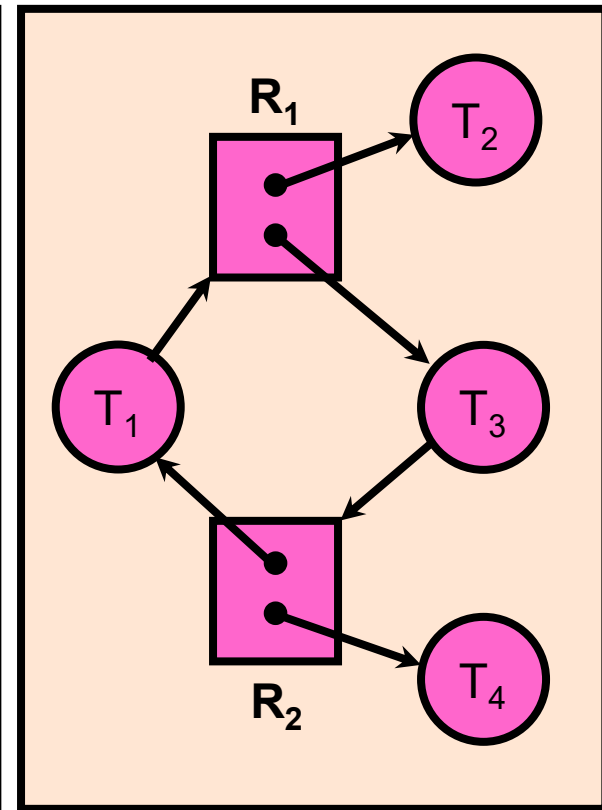
- request edge – directed edge $T_i \rightarrow R_j$
- assignment edge – directed edge $R_j \rightarrow T_i$



Simple Resource
Allocation Graph



Allocation Graph
with Deadlock



Allocation Graph with
Cycle, but No Deadlock

Methods for Handling Deadlocks

- Proactive - Deadlock prevention & avoidance
 - Ensure that the system will *never* enter a deadlock state
- Reactive - Deadlock detection & recovery
 - Allow the system to enter a deadlock state and then recover
- No handling
 - Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX

Dining-Philosopher Problem

- Solutions to the deadlock problem?
- the structure of Philosopher i :

```
do {  
    // think  
  
    wait ( chopstick[i] );  
    wait ( chopstick[ (i + 1) % 5] );  
  
    // eat  
  
    signal ( chopstick[i] );  
    signal ( chopstick[ (i + 1) % 5] );  
} while (TRUE);
```



Dining-Philosopher Problem

- Solutions to the deadlock problem
 - Allow a philosopher to pick up chopsticks only if both chopsticks are available (needs to be done in a critical section)
 - Allow preemption or sharing
 - Add one more chopstick or disallow one philosopher
 - Use an asymmetric solution; an odd-numbered philosopher picks up a left chopstick and then a right one, whereas an even-numbered philosopher picks up in the other way

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously

- **Mutual exclusion**
 - only one process at a time can use a resource
- **No preemption**
 - a resource can be released only voluntarily by the process holding it, after that process has completed its task
- **Hold and wait**
 - a process holding at least one resource is waiting to acquire additional resources held by other processes
- **Circular wait**
 - there exists a set $\{P_1, \dots, P_n\}$ of waiting processes such that P_i is waiting for a resource that is held by P_{i+1} ($0 \leq i \leq n-1$) and P_n is waiting for a resource that is held by P_1 .

Deadlock Prevention

Prevention by removing one of the four necessary conditions

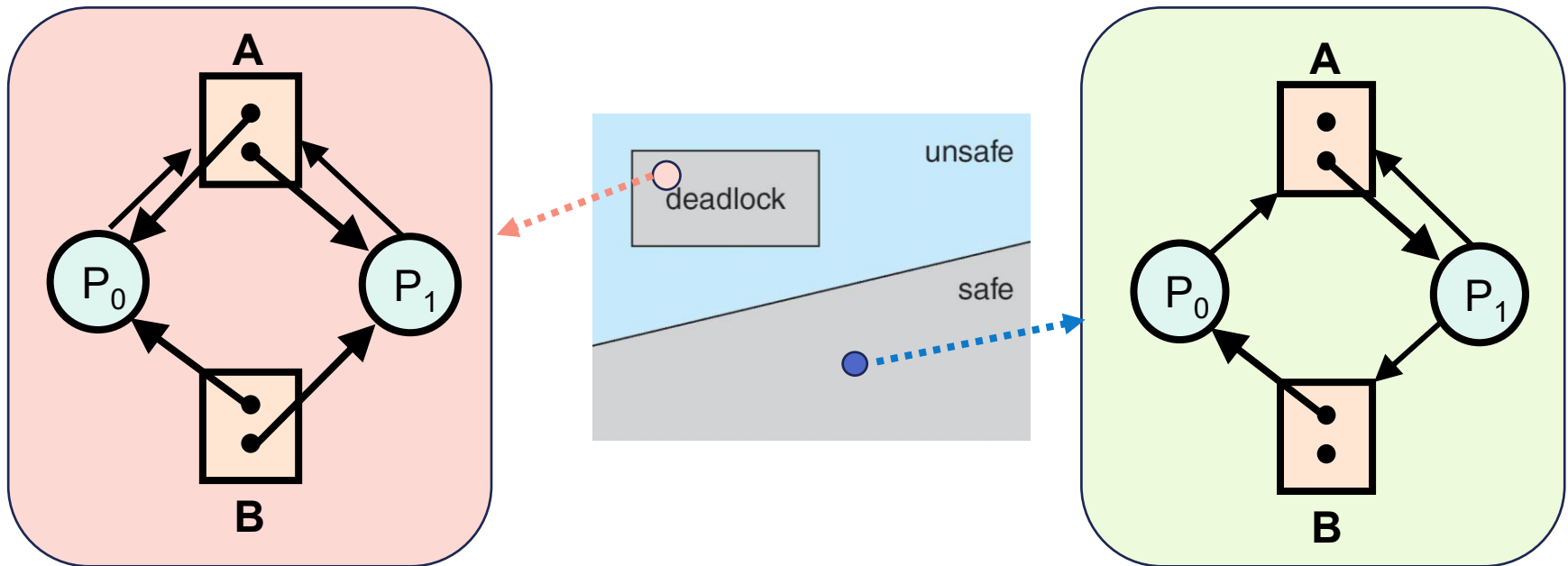
- **Limited resource (i.e., mutual exclusion)**
 - Allow sharing; buy more resources or make them sharable by virtualizing to make “more” copies (e.g., printer spooling).
- **No preemption**
 - Allow preemption with saving/restoring state;
 - CPU preemption between processes; they have copy of registers
- **Hold and wait**
 - Wait on all resources at once (must know in advance)
- **Circular wait**
 - Impose a total ordering of all resources, and processes request resources in an increasing order

Methods for Handling Deadlocks

- Proactive - **Deadlock** prevention & **avoidance**
 - Ensure that the system will *never* enter a deadlock state
- Reactive - Deadlock detection & recovery
 - Allow the system to enter a deadlock state and then recover
- No handling
 - Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX

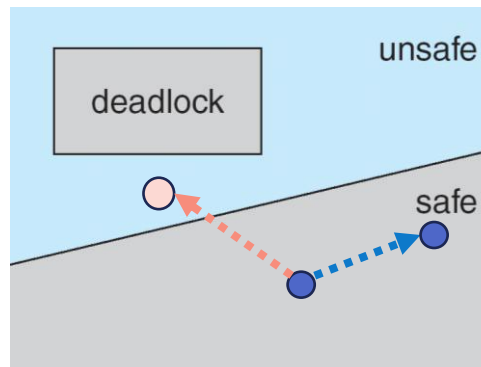
Deadlock Avoidance

- Basic idea (ideal)
 - Divide the world into two states:
 - deadlock-feasible state & deadlock-free state



Deadlock Avoidance

- Basic idea (ideal)
 - Divide the world into two states: deadlock-feasible state & deadlock-free state
 - Determine whether or not the system will go into a deadlock state in future (after some resource assignments)



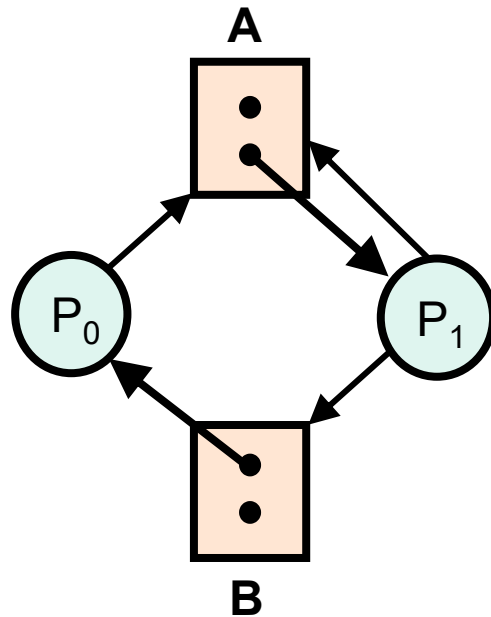
- If so, do something to prevent the system from going into the deadlock state
 - Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state

Safe State

- How can we determine safe states?
 - based on *possible* resource allocation
- Safe state
 - A system is **safe** if there exists any resource allocation sequence $\langle P_1, P_2, \dots, P_n \rangle$ that allows all processes to complete without causing a deadlock
 - i.e., if the system can allocate resources to all the processes **in some specific order** without incurring a deadlock
 - A sequence $\langle P_1, P_2, \dots, P_n \rangle$ of processes represents **a resource allocation order** such that for each P_i , the resources that P_i can still request can be satisfied by the currently available resources + the resources held by all the other P_j , with $j < i$

Safe State

- Safe state
 - A system is **safe** if there exists any resource allocation sequence $\langle P_1, P_2, \dots, P_n \rangle$ that allows all processes to complete without causing a deadlock

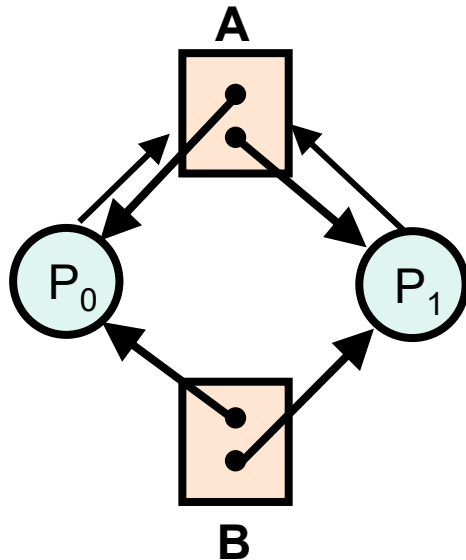


- Is it safe?
 - Yes. For example, we can allocate in the order of $\langle P_0, P_1 \rangle$ to complete all the processes

Safe State

- Safe state

- A system is **safe** if there exists any resource allocation sequence $\langle P_1, P_2, \dots, P_n \rangle$ that allows all processes to complete without causing a deadlock



- Is it safe?

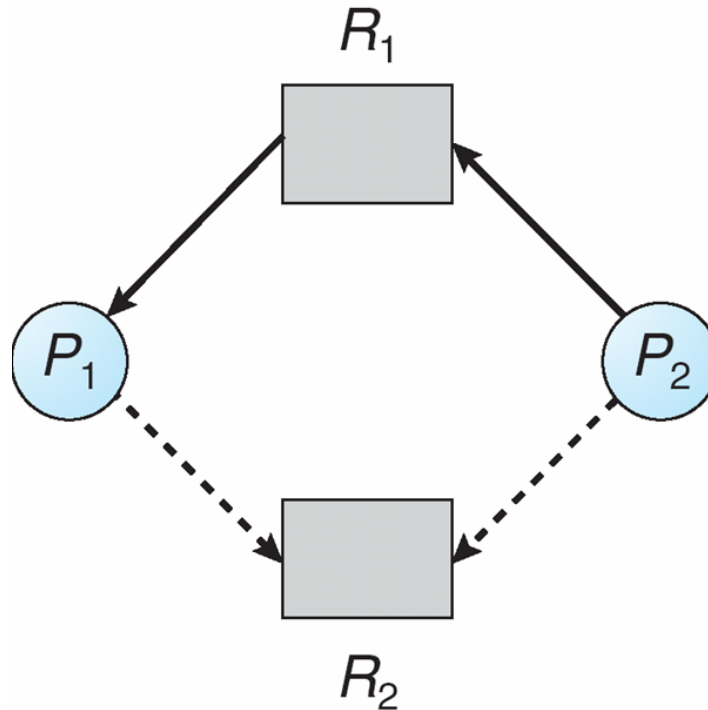
- No. Since there is no possible sequence in which all processes can complete

Avoidance algorithms

- Single instance of a resource type
 - Use a resource-allocation graph
- Multiple instances of a resource type
 - Use the banker's algorithm

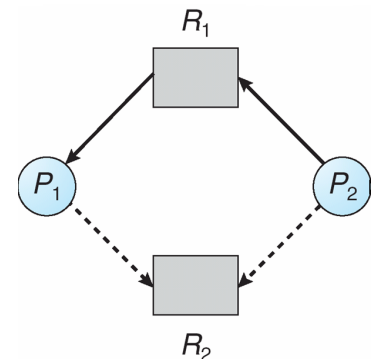
Resource-Allocation Graph

- In addition to **request** and **allocation** edges, we have an additional **claim** edge – a dashed line



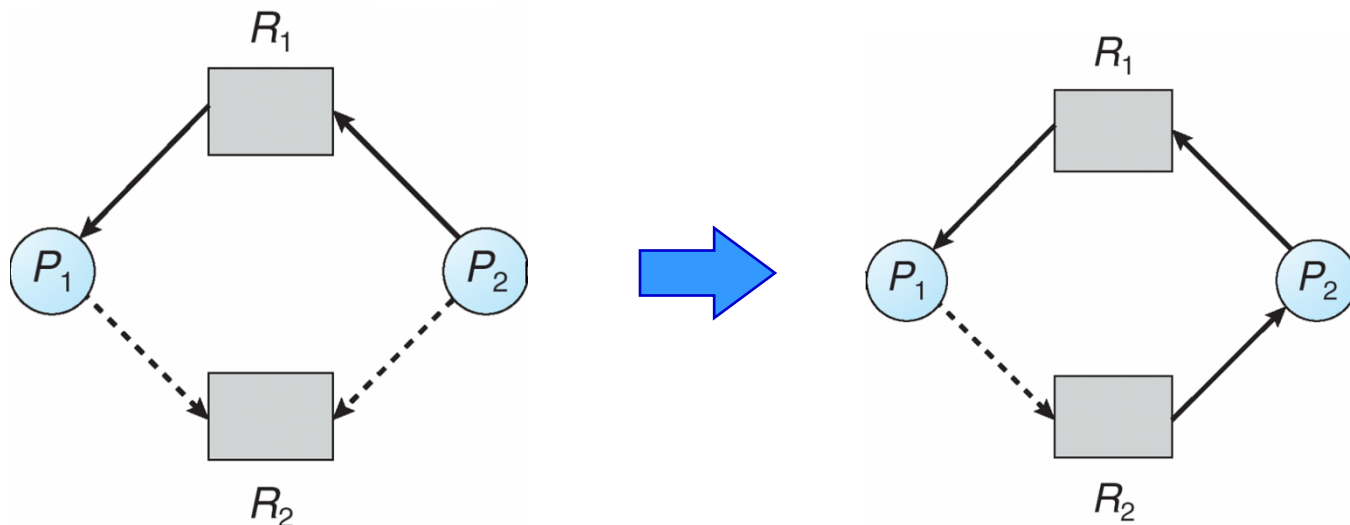
Resource-Allocation Graph

- **Claim edge** $P_i \rightarrow R_j$ indicated that process P_i may request resource R_j ; represented by a dashed line
 - Claim edge converts to request edge when a process requests a resource
 - Request edge converted to an assignment edge when the resource is allocated to the process
 - When a resource is released by a process, assignment edge reconverts to a claim edge
- Resources must be claimed *a priori* in the system

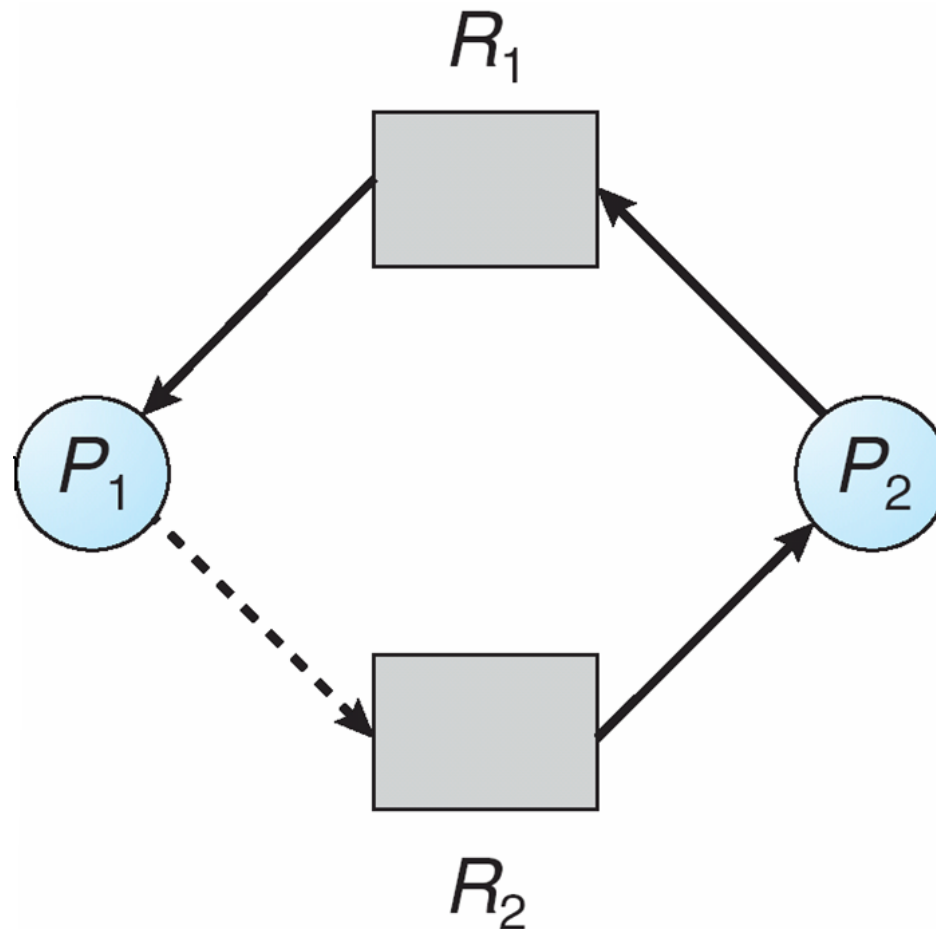


Resource-Allocation Graph Algorithm

- Suppose that process P_2 requests a resource R_2
- The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph



Unsafe State in Resource-Allocation Graph



Avoidance algorithms

- Single instance of a resource type
 - Use a resource-allocation graph
- Multiple instances of a resource type
 - Use the banker's algorithm

Banker's Algorithm

- Banker's algorithm
 - Dijkstra, 1965
 - Works for resources with multiple instances
 - System Model (assumptions)
 - Each process must a priori claim its maximum use
 - When a process requests a resource, it may have to wait
 - When a process gets all its resources, it must return them in a finite amount of time

Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

- **Available:** Vector of length m . If available $[j] = k$, there are k instances of resource type R_j available
- **Max:** $n \times m$ matrix. If $Max[i,j] = k$, then process P_i may request at most k instances of resource type R_j
- **Allocation:** $n \times m$ matrix. If $Allocation[i,j] = k$ then P_i is currently allocated k instances of R_j
- **Need:** $n \times m$ matrix. If $Need[i,j] = k$, then P_i may need k more instances of R_j to complete its task

$$Need[i,j] = Max[i,j] - Allocation[i,j]$$

Safety Algorithm

1. Let *Work* and *Finish* be vectors of length *m* and *n*, respectively.
Initialize:

Work = *Available*

Finish [*i*] = *false* for *i* = 0, 1, ..., *n*- 1

2. Find and *i* such that both:

(a) *Finish* [*i*] = *false*

(b) $Need_i \leq Work$

If no such *i* exists, go to step 4

3. *Work* = *Work* + *Allocation*_{*i*}
Finish[*i*] = *true*
go to step 2

4. If *Finish* [*i*] == *true* for all *i*, then the system is in a safe state

Resource-Request Algorithm for Process P_i

Request = request vector for process P_i .

If $Request_i[j] = k$ then process P_i wants k instances of resource type R_j

1. If $Request_i \leq Need_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim

2. If $Request_i \leq Available$, go to step 3. Otherwise P_i must wait, since resources are not available

3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$Available = Available - Request_i;$

$Allocation_i = Allocation_i + Request_i;$

$Need_i = Need_i - Request_i;$

If safe $\Rightarrow$ the resources are allocated to P_i

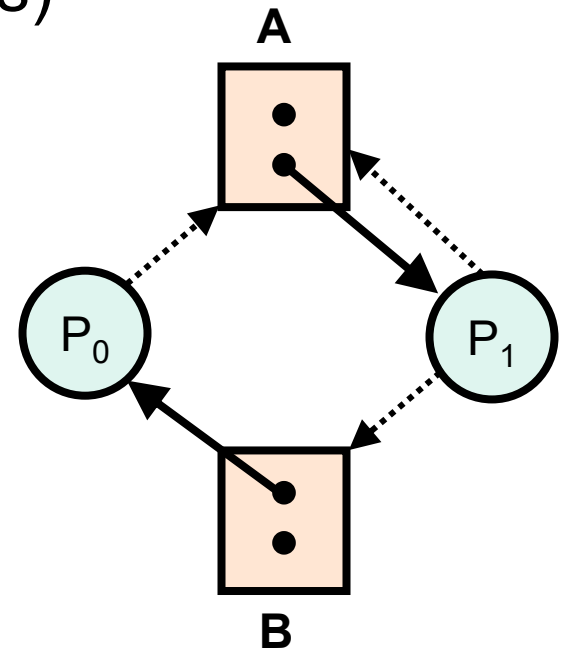
If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored

Example of Banker's Algorithm

- 2 processes: P_0 and P_1 ;
- 2 resource types:
A (2 instances) and B (2 instances)

Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B	A B	A B
P_0	0 1	2 1	1 1
P_1	1 0	2 1	



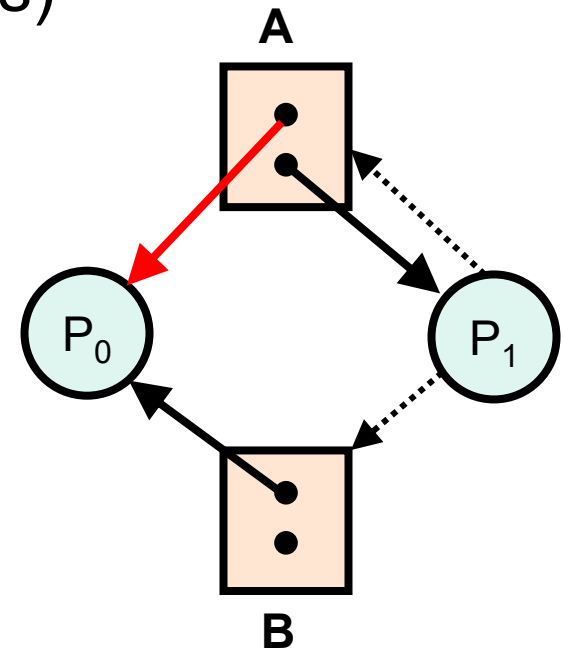
What if request for (1,0) by P_0 is granted?

Example of Banker's Algorithm

- 2 processes: P_0 and P_1 ;
- 2 resource types:
A (2 instances) and B (2 instances)

Snapshot at time T_0 :

	<u>Allocation</u>		<u>Max</u>		<u>Available</u>	
	A	B	A	B	A	B
P_0	0	1	2	1	1	1
P_1	1	0	2	1	0	1



What if request for (1,0) by P_0 is granted?

Example of Banker's Algorithm

- 5 processes P_0 through P_4 ;
- 3 resource types:
A (10 instances), B (5 instances), and C (7 instances)

Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Is the system at time T_0 safe?

Example (Cont.)

- The content of the matrix *Need* is defined to be *Max – Allocation*

	<u>Need</u>		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

- The system is in a safe state since the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies safety criteria

Example: P_1 Request (1,0,2)

- Check that Request $\leq$ Available (that is, $(1,0,2) \leq (3,3,2)) \Rightarrow \text{true}$

	<u>Allocation</u>				<u>Need</u>				<u>Available</u>			
	A	B	C		A	B	C		A	B	C	
P_0	0	1	0	7 5 3	0	0	0		3	3	2	
P_1	2	0	0	3	0	2	3 2 2	0	2	3	0	
P_2	3	0	2	9 0 2	0	0	0		2	1	0	
P_3	2	1	1	2 2 2	0	0	0		5	3	2	
P_4	0	0	2	4 3 3	0	0	0		5	2	1	
					7 4 3	0	0	0		7	4	3
					1	2	2	0 2 0	0	0	0	
					6 0 0	0	0	0		3	1	2
					0 1 1	0	0	0		7	4	5
					4 3 1	0	0	0		0	0	2
										7	5	5
										1	5	5
										10	5	7

Executing safety algorithm shows that
sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies safety requirement

- Executing safety algorithm shows that

sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies safety requirement

Example:

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	3 3 2
P_1	2 0 0	1 2 2	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Can request for (3,3,0) by P_4 be granted?
- Can request for (0,2,0) by P_0 be granted?

Methods for Handling Deadlocks

- Proactive - Deadlock prevention & avoidance
 - Ensure that the system will *never* enter a deadlock state
- Reactive - **Deadlock detection & recovery**
 - Allow the system to enter a deadlock state and then recover
- No handling
 - Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX

Deadlock Detection

- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme

Detection-Algorithm Usage

- When, and how often, to invoke depends on:
 - How often a deadlock is likely to occur?

Detection-Algorithm Usage

- When, and how often, to invoke depends on:
 - How often a deadlock is likely to occur?
 - in the extreme, every time a resource request is not granted
 - whenever CPU utilization drops below, say, 40%

Deadlock Recovery (1)

- Process termination
 - Reclaims all resources allocated to the terminated process
 - Abort all deadlocked processes
 - Abort one process at a time until the deadlock cycle is eliminated

Deadlock Recovery (2)

- Resource preemption
 - Selecting a victim
 - minimize cost
 - Rollback
 - return to some safe state, restart process for that state
 - Starvation
 - same process may always be picked as victim, include number of rollback in cost factor